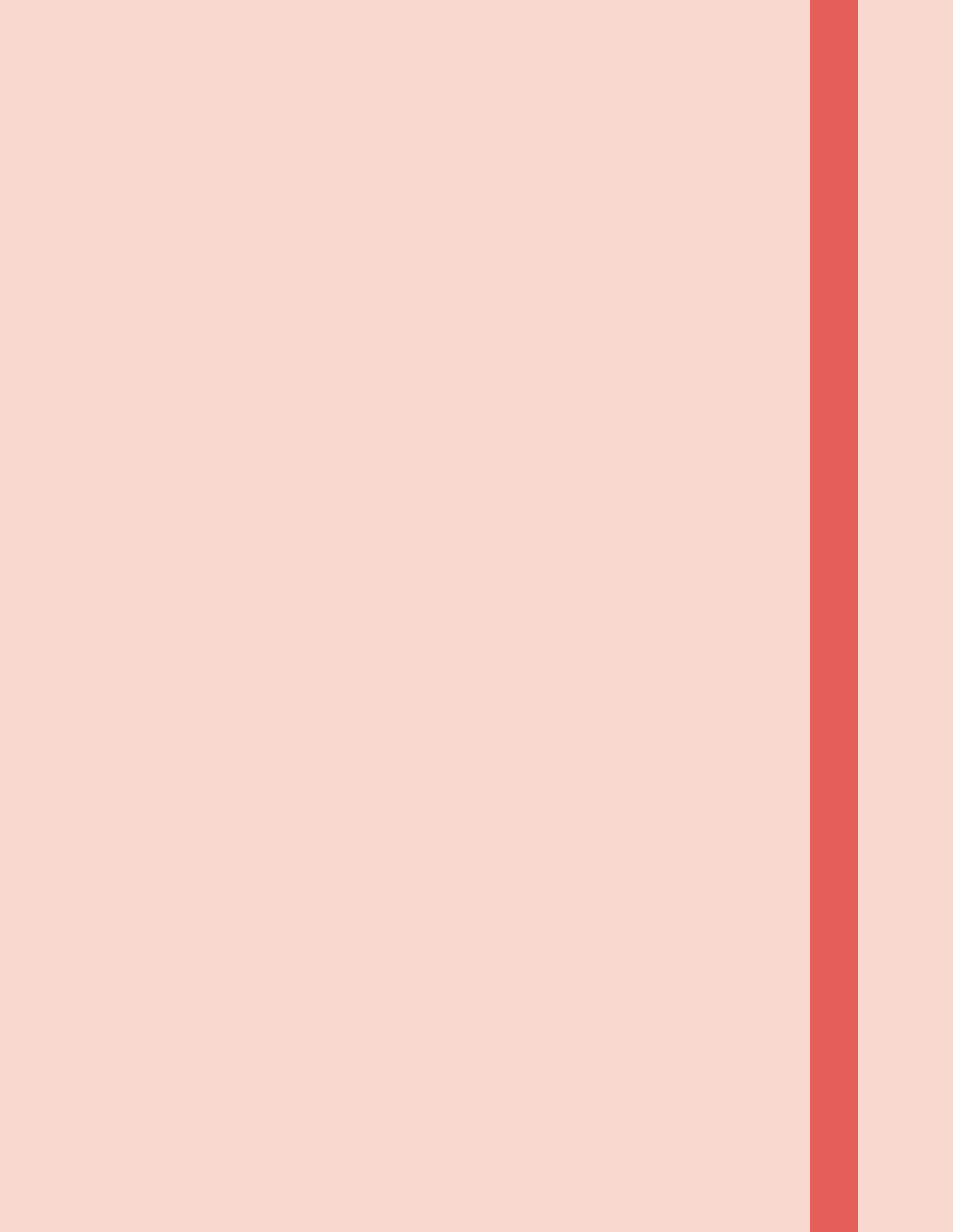




Αντικειμενοστραφής Προγραμματισμός

Γενικά

Ο κώδικας οργανώνεται σε κλάσεις (οντότητες) οι οποίες μπορούν να συνδεούνται με μια σχέση εξειδίκευσης. (Κληρονομικότητα *Inheritance*)

Αφού έχουν οριστεί οι κλάσεις, μπορούμε να ψαξούμε αντικείμενα (*objects*) (έχουν ιδιότητες και συμπεριφορά)

Χρησιότητα - Πλεονεκτήματα

- Εφαρμογές με περίπλοκες οντότητες (προσωπικότητες)
- Τμηματοποίηση κώδικα (κλάσεις)
- Ανεύθυια αναπαράσταση των οντοτήτων / εννοιών του προβλήματος προς επίλυση

Συνέπειες

- Ευκολότερη συντήρηση και συνειρήνηση
- Εύκολα χρησιμοποιημένοι κώδικες
- Εύκολα γνωστοί κώδικες

Programming types

Procedural Programming

1. σειρά/διασπρά διαδικασιών
2. δεδομένα → διαδικασίες / ενέργειες → έξοδος
3. Ποιες είναι οι διαδικασίες που χρειάζονται για την υλοποίηση και αξιολόγηση τους καλύτερους αλγόριθμους
4. τετραγωνική πίνα, linked, sorting
5. Algol 60, Algol 68, FORTRAN, Pascal, C

Modular Programming

1. Τηματοποίηση κώδικα (modules)
 2. Ανίχνευση αλληλοποροπίας (name conflicts)
 3. interfaces (δηλώσεις)
- export** → να ονομάζονται ονόματα ονόματα που γράφονται έξω
- import** → να χρησιμοποιούνται ονόματα που ονομάζονται από άλλα αρχεία

4. module m1
 export void bla1()
 void bla1() {
 ...
 foo();
 ...
 }
 void foo() { ... }

module m2
 export void bla2()
 import void bla1()
 from m1
 void bla2() {
 ...
 bla1();
 foo();
 ...
 }
 void foo() { ... }

5. ανάπτυξη εκτελούσι κώδικα

6. οργάνωση δεδομένων και διαδικασιών

7. πια είναι τα στοιχεία (modules) που μας χρειάζονται + χωρίζουμε το πρόγραμμα να να κρύψουμε τα δεδομένα και τις διαδικασίες στα κατάλληλα modules

8. Module-2, Module 3

9. Στοιχεία διαδικασιών

- επεξεργασία προσωπικού
- επεξεργασία εγγράφων

Object oriented programming

1. σχεδιασμός δεδομένων με συμπεριφορά
2. η συμπεριφορά υλοποιείται μέσω ορισ-
τες διαδικασίες
3. Προσδιορισμός ιεραρχιών τύπων δεδο-
μένων
4. Κοινά χαρακτηριστικά αυτών (κληρονο-
μότητα)
5. Προσδιορισμός κλάσεων → δώσε τους
αετούργιες → εκφράσε τις κοινές μέθω
κληρονομικότητας
6. Shape {color, surface, rotation}

Circle



Triangle



Ε. Simula, C++, Java, ADA, C#,
Modula-3, Smalltalk

Ποια γλώσσα να επιλέξω?

Εξαρτάται από την φύση του προβλήματος

Data abstraction

Εκφραστοντα δεδομένα

τύποι $\leftrightarrow$ λειτουργίες συνδέονται

↳ Δείς ποιους τύπους θες και δώσε
λειτουργίες για τον καθένα

ΑΔΔες χρήσεις OOP

(UML)

→ Αναίρεση και Σχεδίαση λογισμικού

→ Αναπαράσταση γνώσης

→ Βάσεις δεδομένων

Τι υποστηρίζει ο OOP

Αρχικότητα

State

1. Κλάσεις, αντικείμενα, κληρονομικότητα
2. Έλεγχος τύπων
3. Encapsulation (information hiding, τύπος ↔ λειτουργίες, ποια: χαρακτηριστικά / λειτουργίες που καθορίζουν τον τύπο)

ΙΔΙΟΤΗΤΕΣ

Στοιχειώδους

Encapsulation

Reusability

Polymorphism

Υπερφόρτωση
συνόρων (overloading)

Γενικευμένη, τύποι,
διαδικασίες (Generic Programming)

Υποτύποι (subtyping)

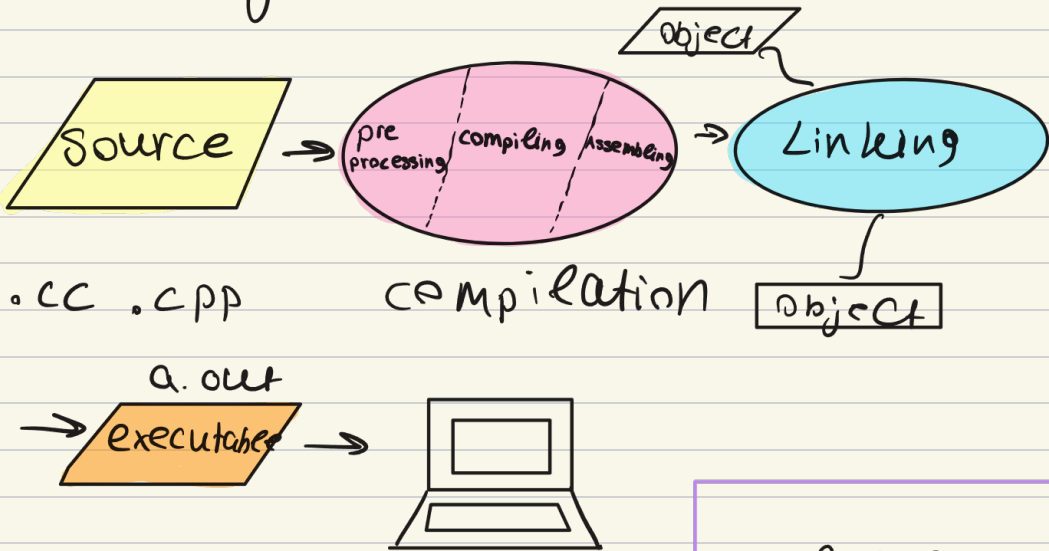
Παραδείγματα information hiding
από το πραγματικό ζώο.

Αλλαγές χαρακτηριστικών (κονομία των
μυών, δεν χρειάζεται βασίλειο)

Τηλεκίνηση (αλλά πατάω ένα
κουμπί)

Ανιστοίχα και αν κάτσει η διαδικασία, δεν θα μας επηρεάσει

Μεταγλώττιση (Unix)



```
#include <iostream>
using namespace std;
```

```
int main() {
```

```
    cout << "Hello world" << endl;
    return 0;
}
```

δηλώνει στον
κομπιλεξέρ να
να ηχητικά βιβλίο
δηλώνει που
βρίσκονται στο
αρχείο-εγκατάσταση
iostream

Τύποι

Τελετοί

bool, char, int, double, enum, void

αριθμητικός



Παραδείγματα: int*, char[], int8, ()

συνάρτηση - postfix

Αριθμητικές σε δέσμες

Δομές δεδομένων + ηλίσες

type casting

structs type def για μετασχηματισμούς
(παραδείγματα)

union

Τελετοί

+, -, *, /, %, <<

Παρατηρήσεις

=> Δεν υπάρχουν ενσωματωμένες συναρτήσεις
και κλήσεις συναρτήσεων

=> Να γράφουμε μηχανικές εκφράσεις με
μηχανικούς τελετοί

=> Να βάλουμε παρενθέςες

=> Η σειρά αντιστοίχης μας έκφρασης δεν είναι σημαντική

Ευρωπές

Scoping

```
if (condition) statement  
if (condition) s1 else s2  
switch (expression) case 1: s1 ...  
case n: sn
```

Επανάληψεις

```
while (expression) { st. }  
do...st...while (expression)  
for (init-st; expression; expression) {  
    Statement  
}
```

Μεταφορές ελέγχου

```
case constant : st( return express; )  
default: st  
break;  
continue;
```

} goto identifier;
↑
να αποβείτα

Σταθερές

- Χρήσιμες για ορίσματα συναρτήσεων
 - Ισχύουν μόνο μέσα στο αρχείο που ορίζονται
- $\text{constant } a = 5;$ (internal linkage)

Το περιεχόμενο του a δεν αλλάζει και γ' αυτό πρέπει να αρχικοποιείται σταθερά αυτοκείμενο

Δήλωση σταθεράς για το αυτοκείμενο στο οποίο δείχνει ο δείκτης $\delta i j$ δεν μπορώ να αλλάξω το

$\text{constant } *pc = \delta i j$ ($*pc = 7$)
Σταθερός Δείκτης

Δήλωση σταθεράς για τον δείκτη

$\text{int } *const\ cp = \delta i j$

δεν μπορώ να αλλάξω το που δείχνει ο δείκτης (δj)

Σταθερός Δείκτης σε Σταθερό Αυτοκείμενο
Δήλωση σταθεράς και για τα δύο

$\text{constant } *const\ cp\ c = \delta i$

ο δείκτης δείχνει πάντα στο ίδια διεύθυνση και η τιμή δεν μπορεί να αλλάξει

Συναρτήσεις

ορίσματα

Γενικά

Δηλώνω: $\text{int sum}(\text{int } i, \text{int } j)$

τύπος επιστ.

ονομα

ορίζεται μόνο μια φορά να έρθει
scope

δεν μπορεί να κληθεί αν δεν έχει
δηλωθεί

inline

λέει στον μεταγλωττιστή να αντικαταστήσει με τον κώδικα της συνάρτησης τις κλήσεις της συνάρτησης, ώστε να χρησιμοποιήσει τον κλασικό μηχανισμό κλήσης συναρτήσεων και επιστροφής αποτελεσμάτων

```
inline int max(int i, int j) {  
    return (i > j ? i : j);  
}
```

Δεν είναι σίγουρο ότι κάθε inline συνάρτηση θα αντικατασταθεί ως inline

Συνθετικά αναδρομικές συναρτήσεις

Υπερφορτώσιον ονομάτων συναρτήσεων

Χρήση του ίδιου ονόματος για διαφορετικές συναρτήσεις

```
void print (int);  
void print (const char*);  
void print (double);
```

Default τιμές ορισμάτων

Επιτρέπουν σε μια συνάρτηση να έχει προεπιγεγμενές τιμές για κάποιες από τις παραμέτρους της.

```
void print (int value, int base=10)
```

Αν καλέσω `print(31)` ο compiler καταλαβαίνει `print(31, 10)`

Αν καλέσω `print(31, 16)` το 10 παραβλέπεται

! Μπορώ να πετύχω το ίδιο αποτέλεσμα με υπερφορτώσιον C function (overloading)

⚠ Το αν η τιμή που δώσαμε ταιριάζει με την τιμή της παραμέτρου γίνεται κατά την δήλωση της συνάρτησης

⚠ Η αντιστροφή της τιμής συμβαίνει κατά την κλήση της συνάρτησης

⚠ Αναγορεύεται να υπάρχουν default τιμές σε ενδιαμέσους ή αρχικά ορίσματα αν αλλιώς θύμουν ορίσματα χωρίς default τιμή

Δοσμευση χωρου στα μνημι

static μεταβλητες → Ο χωρος τους ανατιθεται κατα τη μεταγλωττιση του προγραμματος και διατηρεται καθ' οση τω εκτελεση του.

automatic μεταβλητες → Ο χωρος τους ανατιθεται καθε φορα που γρανουμε στον οριση τους και διατηρεται μεχρι να βγουμε απ' το block που εχει ορισει η μεταβλητη

dynamic memory allocation (heap)

new → δημιουργια αντικειμενων

delete → καταστροφη

? πρεπει να αποδεομενουμε τω μνημι που χρησιμοποιουμε γιατι δευ υηαρηει garbage collection

Πως χρησιμοποιω τους τελεστες new και delete μεσα σε προγραμμα

Για κλάσεις - Αντικείμενα

NEW

```
node* p = new node;
```



δείκτης σε αντικείμενο τύπου node

DELETE

```
delete p;
```

Αν το node είναι κλάση, καλείται ο destructor της

Ειδικά για πίνακες:

```
char* s = new char[10];  
delete [] s;
```

Αν το κάθε στοιχείο του πίνακα είναι αντικείμενο μας κλάσης τότε καλείται ο constructor της

Για αυτές μεταβλητές μπορούμε να
αρχικοποιήσουμε παρά
τη δυναμική δέσμευση

```
node* p = new node(10);
```

delete p;

Τύπος αναφοράς

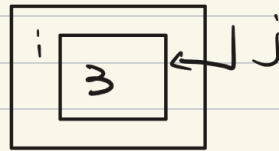
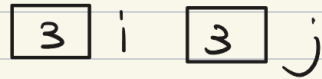
$\langle \text{type} \rangle \& \langle \text{identifier} \rangle = \langle \text{identifier} \rangle$

Εντολές

```
int i = 3;  
int j = i;
```

```
int i = 3;  
int &j = i;
```

Μνήμη



Αναφορές: σταθεροί δείκτες προς το αντικείμενο

Συν C

```
void swap(int *p1, int *p2) {  
    int tmp = *p1;  
    *p1 = *p2;  
    *p2 = tmp;  
}
```

Κάθιστο:

```
int i = 5;  
int j = 10;  
swap(&i, &j)
```

Συν C++

```
void swap ( int& ref1, int& ref2 ) {
```

```
    int tmp = ref1;  
    ref1 = ref2;  
    ref2 = tmp;  
}
```

Κίνηση

```
int i = 5;  
int j = 10;  
swap ( i, j )
```

! Μεγάλα αντικείμενα καλό είναι να τα περνάμε μέσω αναφορών

! Πιο αποδοτικές δηλώσεις συναρτήσεων:

```
void f ( const T& t ) {
```

```
}
```

! Οι πίνακες περνάμε ως αντικείμενα μέσω αναφορών.

Ερβείδιο

↑ η υποφωσφορική του μεταβλητίζεται με αύξηση της οντότητας και έτσι γίνεται με τη δέσμευσή του

Δηλώσεις : ονόμα μεταβλητής - τύπος

`extern int num;` εδώ γίνεται η πραγματική δέσμευση

Ορισμός : δέσμευση ονόματος με συγκεκριμένη οντότητα. (έτσι και δηλώση)

```
char ch; string s;
```

Ανάθεση αρχικών τιμών : βάζουμε τιμές στις μεταβλητές (έτσι και δηλώση) (και ορισμός)

```
int c = 1;
```

! Η ερβείδια μας μεταβλητής ξεκινά από τη δήλωσή της. (υποχρεωτικά πρέπει να δηλωθεί πριν χρησιμοποιηθεί)

! $\Xi \Xi \rightarrow$ ορισμός ερβείδιας

! $\Xi \Xi \Xi \Xi \rightarrow$ επεκτεταμένες ογκώδεις

! Η ερβείδια διευκολύνει την ανάπτυξη του κώδικα

! Σε μια ερβείδια πρέπει να παθε όνομα να έχουμε διαφορετικό ορισμό να να αποφεύγεται η ασάφεια

Εκτός αν έχουμε οντότητες διαφο-
ρετικού είδους.

⇒ η αν στω ίδια εμβέδεια είχαμε
με μεζοβίντι student και με κλόν student
δεν θα υπήρχε θέμα στω C++

⇒ η αν έχουμε δυο συναρτήσεις με
το ίδιο όνομα αλλά να έχουν διαφορε-
τικά ορίσματα (function overloading)

Μεταβλητές

Global

ορίσμος

Ορίζονται έξω από τις συναρτήσεις (σε
ένα αρχείο κώδικα)

εμβέδεια (εξωτερική συνδεσιμότητα)

τα όνομά τους είναι ορατά και προσβάσιμα
από ολόκληρο το πρόγραμμα. Για να χρησιμοποι-
ηθούν σε άλλο αρχείο από αυτό που
ορίστηκαν θα πρέπει να δηλωθούν ως
extern.

lifetime

ήδη η διάρκεια εκτέλεσης του προγράμμα-
τος

ανόητη μανία

Σταθερός χώρος μνήμης

* οι ίδιοι κανόνες ισχύουν και για συναρτήσεις

Static external Variables

Ορισμός: global μεταβλητές (ορίζονται εκτός συναρτήσεων) που προσδιορίζονται static

Scope (εσωτερική συνδεσιμότητα): Η ορατότητα τους περιορίζεται αυστηρά μέσα στο αρχείο που ορίστηκαν και δεν μπορούν να προσεγγιστούν από άλλα αρχεία.

Lifetime: όλη η διάρκεια εκτέλεσης του προγράμματος

Ανθίνευση: στατικός χώρος μνήμης

Static local Variables

ορισμός: ορίζονται ως static μέσα σε μια συνάρτηση και αρχικοποιούνται ταυτόχρονα

scope: είναι ορατές μόνο μέσα σε τη συνάρτηση που ορίζονται

lifetime: όλη η διάρκεια εκτέλεσης του προγράμματος

Ανθίνευση: Στατικός χώρος μνήμης

Βασικά χαρακτηριστικά: διατηρούν την τιμή τους ανάμεσα σε διαφορετικές κλήσεις της συνάρτησης

Automatic Variables τσονικές σε συν.

ορισμός: οι κλασικές μεταβλητές που ορίζονται σε μια συνάρτηση

scope: όλη η συνάρτηση

lifetime: ξεκινάει όταν γίνεται κλήση της συνάρτησης

Αντικείμενα: υφίστανται στο όρισμα εκτέλεσης και ο χώρος τους απελευθεύεται όταν η συνάρτηση ολοκληρωθεί

Τσονικές σε block

ορισμός: μεταβλητές που ορίζονται μέσα σε οποιοδήποτε φάσμα αγκυλών. Ξεκινάει μέσα σε μια εντολή if ή for.

scope: μόνο μέσα στο συγκεκριμένο block

lifetime: διαρκεί όσο εκτελείται ο κώδικας μέσα στο συγκεκριμένο block

Στατικές ταινίες σε block

Ορισμός: μεταβλητή που ορίζεται μέσα σε ένα block αλλά κέρειται προοδωρισμό static

scope: είναι ορατή μόνο σε συγκεκριμένο block

lifetime: Όλη η διάρκεια εκτέλεσης του προγράμματος

Αποθήκευση: Στατικός χώρος μνήμης

Namespaces

Μια επέκταση με όνομα

```
namespace Geometry {
```

```
    const double P1 = 3.14;
```

```
}
```

Εκτός του χώρου

:: → αδειάζει
εξιδύομαι επέκταση

```
exP1 = Geometry::P1;
```

! Μέσα στους χώρους είναι ασφαλές να δηλώνουμε συναρτήσεις και μεταβλητές με εσωτερική συνδεσιμότητα

Εκτός χώρου :

```
using namespace Geometry;
```

→ όχι μέσα σε αρχεία κεφαλαίων

```
double var2 = P1;
```

Ανώνυμος χώρος

→ τα ονόματα είναι ορατά μόνο από το σημείο δύνάμεως τους έως το τέλος του αρχείου (όχι από άλλα αρχεία)

→ χρησιμοποιούνται αντί του προδιοριστικού static

Πώς οργανώνουμε ένα πρόγραμμα που έχει χώρους ονομάτων?

χώρος ονομάτων → αρχείο ετικεταρίδας

ονόματα → αρχείο ημιαίσιου κώδικα → αρχείο κελεταρίδας

! Το αρχείο ετικεταρίδας πρέπει να συμπεριληφθεί σε κάθε αρχείο στο οποίο θέλουμε να χρησιμοποιηθούν ονόματα από τον χώρο ονομάτων.

using namespace std;

```
#include <iostream>
```

```
int main() {
```

```
    std::cout << "yes";  
}
```

```
    n  
#include <iostream>  
using namespace std;
```

```
int main() {
```

```
    cout << "yes";  
}
```

Κλάσεις

struct
class

Οι νέες τύποι που ορίζει ο προγραμματιστής στον οπρ μετατρέπονται ως κλάσεις.

Χρήσιμτητα: τμηματοποίηση κώδικα με ανάπτυξη ή απλοποίηση

! Η πρόσβαση στα μέλη μας κλάσεων γίνεται μέσω member functions (set, get)

! Κάθε αντικείμενο που δημιουργείται κτίζεται από τα data members της κλάσης

! Δημιουργία αντικειμένων → constructors

! Καταστροφή αντικειμένων → destructors

! Τα αντικείμενα δημιουργούνται είτε σαν αυτόματες μεταβλητές, είτε static είτε δυναμικά στον οπρό

Access specifiers - Ανόητη η ήρωα
→ ανόητη η ήρωα

private → ορατά μόνο μέσα στον κλάση που χρησιμοποιούνται (ή από friend functions*)

→ interface

public → ορατά από οποιονδήποτε μέρος του προγράμματος

protected* → ορατά από class που ορίζει να έχουμε και struct ή αν δώσω κλάση -
Από ονόματα

Struct vs class

Struct → αν ένα μέλος δεν δηλωθεί ως private ή public, τότε by default είναι public

χρησιμοποιείται για ανόητους τύπους

class → αν ένα μέλος δεν δηλωθεί ως private ή public, τότε by default είναι private

χρησιμοποιείται για απαραίτητους εώνους δεδομένων, προστατεύει το αντικείμενο

Τεχνική διαμερίσεων

class ονομα Σ
private:

public:

Σ_j

● mutators $\rightarrow$ set (_ _ _)

● accessors $\rightarrow$ get (_ _ _)
Παρά είναι να δηλώνονται ως const (όχι
const)

Δείκτης this

ορίσμός

δείχνει στο ίδιο αντικείμενο για το
οποίο κλήθηκε η συνάρτηση

Πώς λειτουργεί εσωτερικά

τύπος : $x * \text{const this (κλάση } x)$

Όταν γράψουμε object.method(), η

C++ εσωτερικά ηρώνα τη διεύθυνση του αντικείμενου μεθοδ (8 object)

λέει ότι συνάρτηση μεθοδ, ο this παίρνει τη τιμή 8object

Πότε είναι απαραίτητη η χρήση του;

⇒ Όταν μια παράμετρος της συνάρτησης έχει ίδιο όνομα με ένα μέλος-δεδομένου της κλάσης.

```
void set id (int id) {  
    this->id = id;  
}
```

⇒ Όταν μια συνάρτηση πρέπει να επιστρέψει τη διεύθυνση του αντικείμενου. return *this

Δείκτες this σε συναρτήσεις const

```
const X *const this;
```

? Δεν μπορεί να χρησιμοποιηθεί σε στατικές συναρτήσεις-μέλη.

In-line συναρτήσεις

Είναι μια συνάρτηση - μέλος για την οποία ο μεταγλωττιστής προσπαθεί να αυτεκαστοίσει κάθε κλήση της απευθείας με τον κώδικα του σώματός της.

```
class date {
```

```
    int day, month, year;
```

```
public:
```

// inline definition

```
void get (int &d, int &m, int &y) const {
```

```
    d = day;
```

```
    m = month;
```

```
    y = year;
```

```
}
```

→ Παρέχουν ασφάλεια έναντι των μακροεντολών (δεν δημιουργούν παρενέργειες)

→ Έχουν εσωτερική συνδεσιμότητα (ο ορισμός τους πρέπει να είναι διαθέσιμος σε κάθε αρχείο όπου χρησιμοποιούνται)

→ ο προσδιορισμός inline λειτουργεί

ως συμβουλή/υπόδειξη στον compiler

Constructors

Η αρχικοποίηση των αντικειμένων γίνεται μέσω ειδικών συναρτήσεων - μεθόδων που λέγονται constructors

Αυτές οι συναρτήσεις καλούνται αυτόματα όταν ορίζεται μια μεταβλητή του τύπου της κλάσης (όταν δημιουργείται το αντικείμενο)

⚠ Μια συνάρτηση αρχικοποίησης που καλείται χωρίς να έχουν προσδιοριστεί ορίσματα (επεί τους) αναφέρεται ως default constructor

```
class date {
```

```
private:
```

```
    int day;
```

```
    int month;
```

```
    int year;
```

```
public
```

```
    date (int d=1, int m=1, int y=2005);
```

```
};
```

Constructor με λίστα αρχικοποιήσεων

```
classname (type1 v1, type2 v2 ...  
typen vn) : member1(v1), member2  
(v2), ... membern(vn) ≡ ≡
```

παράδειγμα

```
date (int d=0, int m=0, int y=0) : σώμα  
month(m), day(d), year(y) ≡ ≡
```

Πρώτα η αρχικοποίηση και μετά οι
ενσωματώσεις στο σώμα

Μπορώ να δηλώσω έναν constructor
και έχω από τη κλάση αν έχω declar-
ation μέσα στη κλάση (ονομα, παραμετροί)
και definition εκτός κλάσης (κωδικός).
Χρησιμοποιώντας υποχρεωτικά αν τρέξω
εργασία

copy constructor

Μπορούμε να αρχικοποιούμε τα αντικείμενα μας επίσης με αντιγραφή της τρέχουσας αξίας αντικειμένων τους

bitwise copy: αυτόματος μηχανισμός για την αντιγραφή ενός αντικειμένου σε ένα άλλο

τρόπος λειτουργίας

Ο μεταγλωττιστής αντιγράφει το περιεχόμενο των μεθών-δεδομένων του αρχικού αντικειμένου στο νέο, μεταφέροντας 8 bits κάθε μεταβλητής

πότε συγγράφει $x(\text{const } x \& \text{obj})$

1. κατά την αρχικοποίηση ενός νέου αντικειμένου από ένα υπάρχον (copy constructor)

2. κατά τη συνθήκη αμοιβαίας μεταξύ αντικειμένων που έχουν ήδη οριστεί (αυτόματος τελεστής ανάθεσης)

Επαναφορισμός τελεστικής αναθέσεως

deep copy

Όταν μια κλάση χρησιμοποιεί δείκτες ή
δεσφρεύει δυναμικά μνήμη είναι επικίνδυ-
νο το bitwise copy καθώς μπορεί δύο
αντικείμενα να έχουν την ίδια θέση μνήμης

Με τον επαναφορισμό δημιουργούμε ένα
νέο ανεξάρτητο αντίγραφο δεδομένων
χωρίς να προκαλούμε σφάλματα κατά τη
αποδέσμευση μνήμης

Προστασία από τη ταυτοτική αναθέσεως

Μέσω του this μπορούμε να ελέγχουμε
αν ένα αντικείμενο ανατίθεται στον
εαυτό του ($s=s_j$). Αν ο τελεστής απο-
δεσφύση υπάρχει μνήμη πριν αντι-
γράψει τα νέα δεδομένα, με ταυτοτική
αναθέσεως θα κατέστρεφε τα δεδομένα
που ερίδι το αντικείμενο προηγουμένως
δυσβάδει.

Χορηγική αλυσίδα των αναθέσεων

chaining, μοιάζει σαν τους ενσωματωμένους
return * this $\rightarrow a=b=c_j$ αυτού

Παρατηρήσεις

Κατασκευή αυτεκτιμένου κλάσους με
αρχικοποίηση με παραμέτρους αλθου
τινθου → constructor

Κατασκευή αυτεκτιμένου κλάους με
αυτοχρηνη υπάρχουνες αυτεκτιμένου
τες κλάους → copy constructor

Ανάθεση τιμων σε υπάρχουντα αυτεκτιμε-
να → overloaded = operator

! η χρñου του τελεστυ ανάθεσης

στον ορισμό αυτεκτιμένου δεν είναι
έκφραση ανάθεσης. Υπονοει χρñου του

copy constructor

Destructors

Για ασφαλή απελευθέρωση χώρου και καταστροφή Αντικειμένου

Σύνταξη: ^{ομπή που πακί} ~ classname

[↑] όνομα κλάσης

! Βγαίνοντας από το σώμα της μεθόδου δεν σταματά η εκτέλεση : Καλούνται και οι συναρτήσεις καταστροφής όλων των αντικειμένων που δημιουργήθηκαν (εκτός από αυτών στο heap)

Καταστροφή με τη αντίστροφη σειρά

Ορισμός κλάσεων συντονικά

```
class ΟΜΥΡΑΝΤΙΔΗΣ {  
    private  
        // data members  
    public  
        // member functions  
        // constructors  
        // destructors  
};
```

Σταθερές - κλάσεις

- CONST:
- **Συνεργείες πειν** Δεν μεταβάλλουν το αυθεντικό νόημα σε ορισμένα πράγματα (get)
 - **data members** Ανατίθενται εξαρχής και δεν αλλάζουν (δεν μπορεί να χρησιμοποιηθεί συντακτικό set, μόνο μέσω λίστας ορισμών)

Objects
βιώνουν

δεν επιτρέπεται να μετα-

Σταθερά μέλη κλάσεων

Συναρτήσεις και σταθερές μεταβλητές που ανήκουν στην εμβέλεια της κλάσης αλλά δεν ανήκουν σε κάποιο συγκεκριμένο αντικείμενο, δηλαδή εκτελείται χωρίς να πρέπει να υπάρχει κάποιο αντικείμενο

! Ενώ κάθε αντικείμενο έχει το δικό του αντίγραφο για τα κοινά μέλη, για τα σταθερά μέλη υπάρχει μόνο ένα αντίγραφο στην μνήμη για ολόκληρη την κλάση

Χρήση

όλα αντικείμενα «fουν» ανάμεσα στην

! Δεν μπορούμε να χρησιμοποιήσουμε τον δείκτη this σε μια static συνάρτηση - μέλος μιας κλάσης

Δεν μπορούμε να δηλώσουμε μια static συνάρτηση με `const`

→ για επεξεργασία κλάσεων

Class composition

Όταν μια κλάση έχει ως μέγιο ένα αντικείμενο (has a) άλλης κλάσης.

Η σύνθετη κλάση έχει ποσοβαρύνει στα public members των κλάσεων που έχει ως μέγιο

class RNA { και μέσα αυτό

Gene g; → composition

int seq;

Σ_j

↑
πρωτα αρχικοποιείται ως

Οπότε, τι έχουμε εγχείρησες έχουμε όταν πρέπει να συνεισφέρει μια κλάση σε μια άλλη

1. Μέθoς τύπου κλάσης (δημογραφία αυτώνειμένου)

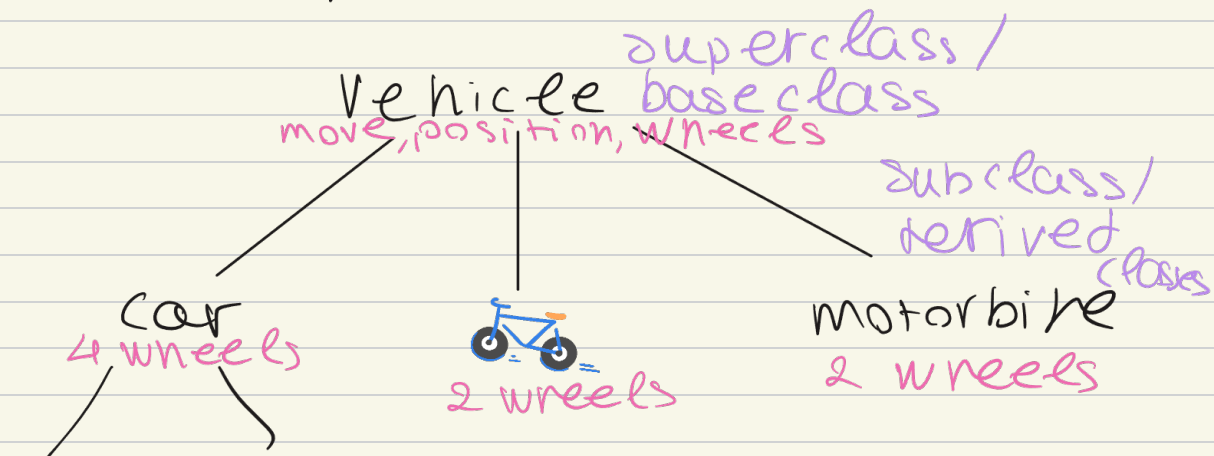
2. Μέθoς τύπου Αναφοράς &

3. Μέθoς τύπου δείκτη πορεί να υπάρχει ή όχι

! Όταν χρησιμοποιούμε δείκτη, υπάρχει κίνδυνος να μη δείχνει σε αληθινό (run time error)

Κληρονομικότητα

→ για evaluation
minou κλάσεων



cabrio limousine

no roof roof → είδος διαφοράς

Οι κλάσεις car, bike, motorbike κληρονομούν ότι έχει ορίζει ο πεν κλάση vehicle

taxonomy: ιεραρχική οργάνωση εννοιών ή οντοτήτων σε σχήμα που μοιάζει με δέντρο

multiple inheritance: αν μια κλάση αποτελεί υνολόγηση περισσότερων κλάσεων από μια

ένας παραγίγος είναι και πτωκό και φίλο

multi level inheritance η κλάση B
έχει υποκλάση της A και η < υποκλά-
ση της B από και της A

Κληρονομικότητα $\rightarrow$ is-a

Συνθεση $\rightarrow$ has-a

Διότι η κληρονομικότητα:

```
class B : public A {
```

```
}
```

Διότι συνθετική κληρονομικότητα

```
class C : public A, public B {
```

```
}
```

Υπευθύνιση - ορατότητα μελών

private μέλη → εμβέδεια κλάσσης/friend functions

protected → εμβέδεια κλάσσης, φίλικες συναρτήσεις, εμβέδεια παραγόμενων κλάσεων, φίλικες συναρτήσεις, φίλικες κλάσεις

public εμβέδεια προγράμματος

Παρατηρήσεις

♥ Πρώτα καλείται ο constructor της base class και μετά ψτίζεται το αντικείμενο της derived

♥ Οι constructors δεν μπορούν ποὺντα να χρησιμοποιούν resources

♥ Σε copy constructors : πρέπει σὺν δίσα

αποκρίνεται να καλέσουμε τη συνάρτηση αντιγραφής της βασικής κλάσης, ξεχωρίζοντας το αντικείμενο - πρότυπο να αντιγραφεί σωστά η φονεική θέση του αντικειμένου

→ αλλαγή τιμών μεταξύ δύο αντικειμένων που υπάρχουν ήδη στην μνήμη

♥ Assignment operator: μέσα στο σώμα της συνάρτησης πρέπει να καλέσετε ρητά τον τελεστή αλλαγής της βασικής κλάσης, αλλιώς τα κληρονόμημένα μέλη θα μείνουν με τις παλιές τους τιμές

♥ Ένα αντικείμενο μιας υποκλάσης διαθέτει όλα τα χαρακτηριστικά των υπερκλάσεων του

♥ Συντρέχει ένας δείκτης ή αναφορά στον υπερκλάσης να δείχνει σε αντικείμενο υποκλάσης

♥ Αντί δεν συντρέχει η αντίθεση αντικείμενου υπερκλάσης σε αντικείμενο υποκλάσης.

♥ Ρητό cast: μπορεί ο μεταγλωττιστής να υπερμεταφράσει έναν δείκτη υπερκλάσης ως δείκτη υποκλάσης (down casting)

friend functions

Δεν αποτελεί member function μιας κλάσης αλλά έχει πρόσβαση στα **private** και **protected** μέλη της

Δεν διαθέτει δείκτη this

Virtual functions

Μια member function που δηλώνεται σε μια base class και την οποία περιμένουμε να επαναορίσουν (override) οι **παράγωγες κλάσεις**

Σκοπός: εξεύρεση συμμορφισμού, εξυπνούν στον προγραμματιστή να χειρίζεται αντικείμενα **διαφορετικών υποκλάσεων** μέσω δεικτών ή αναφορών της βασικής κλάσης

Λοιπές συναρτήσεις (non-virtual)

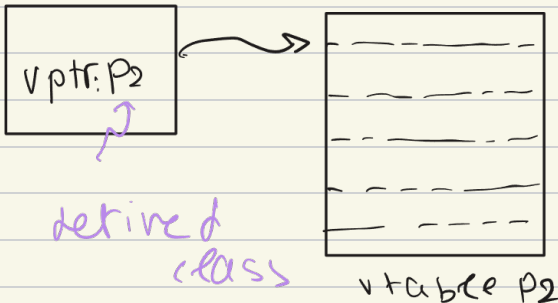
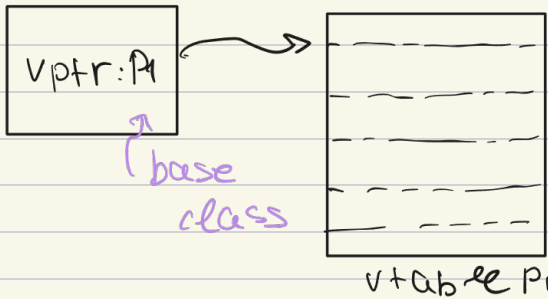
Οποιοσδήποτε θα επιλεγεί και απορρίπτεται κατά την **μεταγλώττιση**

Virtual

το νηος οποιος θα εκτελεστεί καθορίζεται κατά την εκτέλεση και εξαρτάται αν τον πραγματικό τον του αντικείμενου στο οποίο αναφέρεται ο δείκτης ή η αναφορά

Virtual table

virtual functions $\rightarrow$ ο μεταγλωττιστής παράγει virtual table



virtual destructors

Σημαντικοί μαζί όταν διαγράφουμε ένα αυτεινόμενο derived class μέσω δείκτη, διασφαλίζουμε ότι θα κληθεί σωστά η συνάρτηση καταστροφής της υποκλάσης αποτρέποντας memory leaks

! Δεν υπάρχουν virtual constructors

pure virtual functions

Μπορώ με συνάρτηση να εν δηλώσω pure virtual βαζοντας $= 0$ στο τέλος της δηλώσεως

Μετα τέτοια συνάρτηση δεν έχει υλοποίηση οπότε βασική ηλίσση και κάνει τη βασική ηλίσση abstract class



Δεν μπορώ να δηλώσω αντικείμενα από αυτή

Όταν μια κλάση κληρονομεί από μια abstract, τότε "υποχέεται" στον μεταγλωττιστή ότι θα δώσει σώμα και κώδικα σε αυτές τις pure virtual functions.

Αν δεν οριστούν όλες: Αν η κλάση υπονομιώσει μερικές από αυτές αλλά αφήσει έστω και μια χωρίς υπονομιση, τότε η κλάση αυτή παραμένει abstract

Ενώ **δεν** μπορούμε να ορίσουμε αντικείμενα μπορούμε να ορίσουμε **δείκτες και αναφορές** σε abstract classes.